Savitribai Phule Pune University
Centre For Distance and Online Education

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

CENTRE FOR DISTANCE AND ONLINE EDUCATION

Program	**Master of Computer Application**		
Course	**Programming from First Principles**		
Topic Name	**Practical Session Part - 2 Binary Search**		
Topic Code	**-**		
Unit/Module No. & Name	**12**		**Recursion**
Self-Learning Hours	**-**		

Topic Details

© SPPU | Original Content | All Rights Reserved Copying or Reposting Without Permission is Strictly Prohibited

OBJECTIVE

1. This unit explains binary search in simple English with an academic tone for undergraduate students.
2. The main purpose is to help learners understand how binary search works, why it is faster than a simple linear search in many cases, and where it is used in computing.
3. Binary search is an important searching method because computers often need to find one item from a large ordered collection.
4. When the data is already sorted, binary search can reduce the amount of work by checking the middle value and cutting the remaining search area into half.
5. After studying this unit, students should be able to define binary search, state its conditions, explain its step by step process, and solve basic problems using arrays, lists, or ordered records. Students should also understand the difference between binary search and linear search.
6. Binary search is taught in data structures, algorithms, and programming courses. It helps students see how logical design improves performance.
7. The topic also builds a foundation for trees, databases, indexing, and more advanced search techniques.
8. In real systems, searching is a daily operation. Computers search names, marks, product codes, phone numbers, records, dates, and stored files. Binary search is useful when the data is arranged in order and many searches must be done quickly.

1.0 INTRODUCTION

1.1 Why Searching Matters

1.1.1 Role Of Searching In Computing

Searching is one of the most common tasks in computer science. A program may need to find a number in a list, a word in a dictionary file, a student record in a database, or a product code in a store system. If the item is not found quickly, the whole task becomes slow.

1.1.2 Link With Daily Life

Searching also appears in daily life outside computing. A person may look for a name in an attendance sheet, a chapter in a book, or a word in a paper dictionary. If the list is random, the person may check one item after another. If the list is ordered, the person can move more intelligently. Binary search copies this intelligent behavior in a formal and exact way.

1.2 Need For Efficient Searching

1.2.1 Limits Of Simple Search

The easiest search method is linear search. In linear search, the program checks items one by one from the beginning until the target is found or the list ends. This method is simple and useful, especially for unsorted data, but it may take a long time for large lists. If a list has thousands or millions of items, checking each item one by one becomes costly.

1.2.2 Advantage Of Smarter Design

Binary search improves the process by using order. Instead of checking every item, it compares the target with the middle value. If the target is smaller than the middle value, the program knows the target can only be in the left half. If the target is larger, it can only be in the right half. This repeated halving greatly reduces the number of checks.

1.3 Main Idea Behind Binary Search

1.3.1 Search By Halving

The central idea of binary search is halving the search space. Each step removes one half of the remaining list from consideration. Because the search area becomes smaller very quickly, the

method reaches the answer in fewer steps than linear search on large sorted lists. This is why binary search is often presented as a classic example of algorithmic efficiency.

1.3.2 Logical Elimination

Binary search is not only about finding the correct position. It is also about eliminating impossible positions. When the middle value is compared with the target, one whole side becomes unnecessary.

1.4 Educational Importance

1.4.1 Building Computational Thinking

Binary search helps students build computational thinking. They learn that data structure matters, conditions matter, and performance matters. The method shows that a problem can often be solved more quickly when the input has the correct form.

1.4.2 Building Confidence With Indices

Binary search also helps students work with indices such as low, high, and mid. These ideas are important in arrays, loops, recursion, and range handling. Students who understand binary search often become more confident in tracing program flow and writing accurate code.

1.5 Historical And Conceptual Value

1.5.1 Classic Algorithm Status

Binary search is often described as a classic algorithm because it is simple in idea but strong in effect. It shows how a small change in strategy can create a large gain in performance.

1.5.2 Broader Learning Value

The topic teaches more than searching. It teaches ordered thinking, boundary control, decision making, and correctness. Students learn to ask useful questions such as: Is the data sorted? Which half should be ignored? When should the process stop? These questions support good programming habits.

2.0 MEANING AND DEFINITION

2.1 Meaning Of Binary Search

2.1.1 Simple Meaning

Binary search is a searching method used to find a target value in a sorted list by repeatedly checking the middle element and then removing one half of the remaining list from the search. At each stage, the current range is divided into a left half and a right half. One half is kept, and the other half is removed from further checking.

2.1.2 Everyday Interpretation

A common everyday example is searching for a word in a printed dictionary. Since the words are in alphabetical order, a reader does not start from the first page every time. Instead, the reader opens near the middle, checks whether the target word comes before or after that page, and then moves to the correct half. This practical behavior reflects the logic of binary search.

2.2 Formal Definition

2.2.1 Standard Definition

Binary search is a comparison based searching algorithm that works on a sorted collection. It compares the target with the middle element of the current search interval. If the target equals the middle element, the search ends successfully. If the target is smaller, the search continues in the left subinterval. If the target is larger, the search continues in the right subinterval. The process repeats until the target is found or the interval becomes empty.

2.2.2 Core Condition

The most important condition for binary search is that the data must be sorted in advance. Without sorted order, the method cannot correctly decide which half to remove. Many student mistakes happen because they apply binary search to unsorted data.

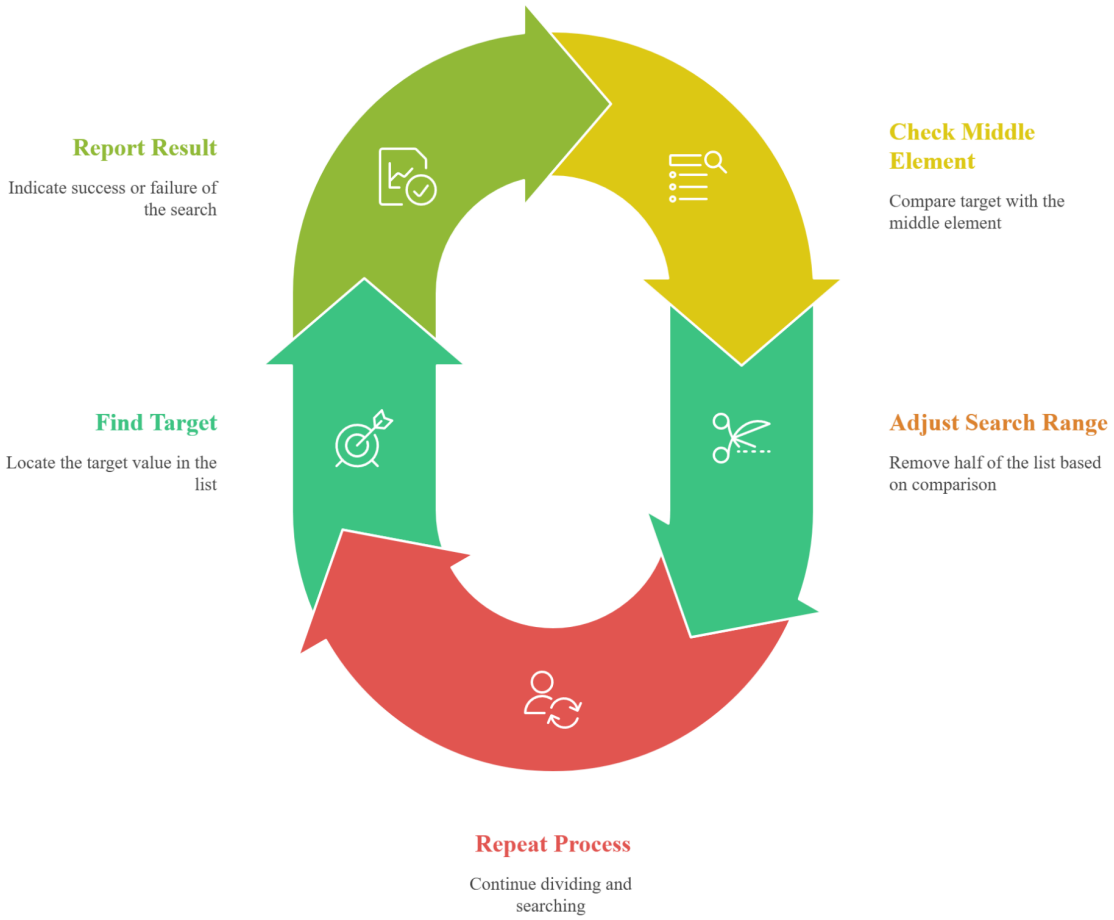
2.3 Main Components Of Binary Search

2.3.1 Low, High, And Mid

Binary search commonly uses three positions. The low index marks the beginning of the current search range. The high index marks the end of the current search range. The mid index points to the middle position. By comparing the target with the value at mid, the program decides whether to move low upward or high downward.

Fig. 1

Binary Search Cycle



This visual illustrates binary search as a continuous cycle, showing finding the target, checking the middle element, adjusting the search range, repeating the process, and finally reporting success or failure.

2.3.2 Comparison Rule

The comparison rule is simple but powerful. If target equals middle value, return success. If target is less than middle value, search the left half. If target is greater than middle value, search the right half. This rule is repeated until the range becomes too small to continue.

2.4 Types Of Implementation

2.4.1 Iterative Definition

In the iterative version, a loop is used. The program updates low and high until the target is found or the condition low is greater than high becomes true.

2.4.2 Recursive Definition

In the recursive version, the function calls itself with a smaller left or right range. The search stops when the target is found or when the range becomes invalid. This version is useful for teaching because it highlights repeated problem reduction.

2.5 Meaning Of Search Success And Failure

2.5.1 Successful Search

A successful binary search ends when the middle element is equal to the target. The algorithm then returns the position or confirms that the value exists. In many programs, the returned position is important because later operations use that index.

2.5.2 Unsuccessful Search

An unsuccessful search happens when the target does not appear in the list. In such a case, the range becomes empty. This means low moves beyond high, and no valid interval remains. The algorithm then reports failure, often by returning a special value such as minus one.

2.6 Complexity Meaning

2.6.1 Time Complexity

Binary search usually runs in O(log n) time. The reason is that each comparison cuts the remaining search area roughly in half. If a list has sixteen elements, the answer can be reached in

about four comparisons in the ideal count pattern. As the list grows, the number of extra steps grows slowly.

2.6.2 Space Complexity

The iterative form of binary search usually uses O(1) extra space because it only needs a few variables such as low, high, and mid. The recursive form uses additional call stack space, often O(log n).

2.7 Correctness Conditions

2.7.1 Sorted Order

Correctness depends first on sorted order. If the data is not sorted, binary search may ignore the side that actually contains the target. Therefore, sorted input is necessary for the result to be reliable.

2.7.2 Proper Boundary Update

Correctness also depends on updating boundaries carefully. If low, high, or mid is handled wrongly, the algorithm may miss the target or enter an endless loop. Good implementation requires both idea and detail.

2.8 Difference From Linear Search

2.8.1 Process Difference

Linear search checks each item in sequence. Binary search checks one item at a strategic position and then removes half of the remaining options. This makes binary search more efficient on sorted data, especially when the list is large.

2.8.2 Input Requirement Difference

Linear search can work on unsorted data, but binary search cannot. This means the better algorithm is not always the correct one. Students must first examine the input condition before choosing a search method.

3.0 NATURE OR TYPE

3.1 Nature Of Binary Search

3.1.1 Decision Based Nature

Binary search is decision based. Every step makes a choice between two possible halves. The method does not move randomly through the list. It uses comparison to decide where the target can still exist. This makes it a highly logical algorithm.

3.1.2 Order Dependent Nature

Binary search is strongly dependent on order. Its power comes from the sorted arrangement of data. Because order guides the decision at every step, the method cannot operate correctly without this structure. This dependence shows that good algorithms often rely on good data organization.

3.2 Structural Nature

3.2.1 Interval Reduction Nature

The search works by reducing an interval. At the beginning, the interval covers the whole list. After each comparison, the interval becomes smaller. This reduction continues until the target is found or no interval remains. The shrinking interval is the structural heart of the algorithm.

3.2.2 Boundary Controlled Nature

Binary search is controlled by boundaries. The low and high indices define the active part of the list. The algorithm changes only these boundaries instead of moving every element. This makes the method efficient and easy to analyze.

3.3 Types Of Binary Search

3.3.1 Iterative Binary Search

This type uses a loop. It is common in practical programming because it is efficient in memory use and simple to test. Students often learn this version first when they study arrays and loops.

3.3.2 Recursive Binary Search

This type uses repeated function calls. It is useful for understanding problem reduction and elegant algorithm design. Although it may use more memory because of the call stack, it clearly shows the repeated nature of the process.

Fig. 2

Binary Search Cycle

This diagram presents the binary search process step-by-step, including initializing boundaries, calculating midpoint, comparing values, adjusting the search interval, and repeating until the target is found or terminated.

3.4 Comparative Nature

3.4.1 Faster Than Linear Search On Sorted Data

Binary search is usually much faster than linear search when the data is sorted and large. This is because binary search reduces the number of comparisons sharply. Even for huge lists, the required comparisons remain relatively small.

3.4.2 Not Universally Better

However, binary search is not always better in every situation. If the data is unsorted and only one search is needed, sorting first may take more time than a simple linear search. This reminds students that algorithm choice depends on context.

3.5 Mathematical Nature

3.5.1 Logarithmic Growth

The behavior of binary search reflects logarithmic growth. Each step halves the problem size. This kind of growth appears in many areas of computer science and mathematics. Understanding it helps students read complexity analysis with more confidence.

3.5.2 Exact Comparison Logic

Binary search uses exact comparison logic. Every choice is based on whether the target is smaller than, equal to, or greater than the middle value. This makes the method easy to reason about and prove correct.

3.6 Data Type Nature

3.6.1 Works On Comparable Data

Binary search can work on numbers, words, dates, and many other data types, as long as the values can be compared and the collection is sorted. This broad use makes the method versatile.

3.6.2 Often Used With Arrays

The method is often taught with arrays because arrays support direct access to the middle element. Random access makes the search fast and natural. This is why arrays and binary search are closely linked in basic algorithm teaching.

3.7 Variational Types

3.7.1 Standard Exact Match Search

The most common type looks for one exact target value. It returns whether the value exists and often gives its index. This is the form most students meet first.

3.7.2 Modified Boundary Search

A modified form can find the first occurrence, last occurrence, lower bound, or upper bound in ordered data with repeated values. These variations are important in advanced problem solving and database style tasks.

3.8 Conceptual Learning Types

3.8.1 Manual Tracing Type

Teachers often use a small sorted list and ask students to mark low, high, and mid by hand. This helps students see how the interval changes after each comparison.

3.8.2 Program Implementation Type

In programming practice, students write code for iterative or recursive search. This type focuses on conditions, loops, functions, and return values. It turns a conceptual method into a working solution.

4.0 EXAMPLES AND CASE STUDIES

4.1 Basic Worked Example

4.1.1 Searching In A Small List

Consider the sorted list: 2, 4, 6, 8, 10, 12, 14. Suppose the target is 10. At the beginning, low points to 2 and high points to 14. The middle element is 8. Since 10 is greater than 8, the left half is removed. The new active range becomes 10, 12, 14. Now the middle element of this range is 12. Since 10 is smaller than 12, the right side is removed. The remaining value is 10, so the target is found. This example shows the step by step halving process clearly.

4.1.2 Learning Point

Students should notice that the method did not inspect every value. It reached the answer by using order and logic. This is the key educational value of binary search.

4.2 Example Of Unsuccessful Search

4.2.1 Target Not Present

Take the sorted list: 3, 7, 9, 15, 21, 30, 42. Suppose the target is 10. The middle value is 15. Since 10 is smaller than 15, the search moves left to 3, 7, 9. The new middle is 7. Since 10 is greater than 7, the search moves right to 9. Since 10 is greater than 9, no valid interval remains. The algorithm reports that 10 is not present.

4.2.2 Academic Value

This example is important because searching is not always successful. A correct algorithm must also know how to stop and report absence properly. Good programming is not only about success cases.

4.3 Example With Words

4.3.1 Alphabetical Search

Consider the sorted list: apple, banana, grape, mango, orange, pear, plum. Suppose the target is mango. The middle word is mango, so the target is found in one step. If the target were grape, the first middle word would be mango, which is later in alphabetical order. Therefore, the search

would continue in the left half. This example shows that binary search works with words as well as numbers.

4.3.2 Broader Data Lesson

Students should understand that binary search does not depend on numeric data only. It depends on comparable order. Any ordered records can use the same logic.

4.4 Case Study One

4.4.1 Student Roll Number Search

A college stores student records by roll number in sorted order. When an office worker enters a roll number, the system must quickly check whether the record exists. Since the roll numbers are ordered, binary search becomes a suitable method. It reduces delay, improves service speed, and helps the office handle many requests in less time.

4.4.2 Institutional Benefit

This case study shows that algorithm choice affects administration. A better search method saves time for staff and students. It also reduces system load when many searches happen each day.

4.5 Case Study Two

4.5.1 Product Code Lookup

A business stores product codes in sorted order inside a processing module. During billing or stock checking, the software must verify a code quickly. Binary search can perform this task efficiently when the code list is ordered. The method helps reduce waiting time in practical business operations.

4.5.2 Business Value

This example shows that binary search supports accuracy and speed in commercial systems. A fast lookup process improves workflow and customer experience. Small algorithm choices can create large practical benefits.

4.6 Case Study Three

4.6.1 Search In Research Data

A researcher has a sorted list of participant identification numbers. During data cleaning, the researcher must check whether certain IDs are already present in the record. Binary search helps verify these IDs quickly. This reduces repeated checking and supports cleaner data management.

4.6.2 Research Value

The case study shows that searching methods are useful in academic research, not only in software companies. Good data handling depends on both organization and efficient algorithms.

1. The method checks the middle value and removes one half.
2. Low, high, and mid are the key control positions.
3. The iterative form uses very little extra memory.
4. Careful boundary updates prevent wrong answers and endless loops.

5.0 Keyword Table

Keyword	Meaning
Binary Search	A search method that repeatedly checks the middle value in sorted data
Search Interval	The current active part of the list being searched
Sorted Data	Data arranged in increasing or decreasing order
Low Index	The starting position of the active range
High Index	The ending position of the active range
Mid Index	The middle position used for comparison
Linear Search	A method that checks items one by one
Iterative	A process that repeats by using a loop
Recursive	A process that solves a problem by calling itself
Time Complexity	A measure of how running time grows with input size
Logarithmic	Growth based on repeated halving
Target Value	The item that the search is trying to find